

Hawkeye: Seeing One Layer Deeper — A Cohesion-Aware Structural Channel for Temporal Link Prediction

Anonymous Author(s)
Anonymous Institution
anonymous@example.com

ABSTRACT

State-of-the-art temporal-link-prediction (TLP) models are, in essence, multi-channel information aggregators: they combine an interaction-history channel, a time-encoding channel, and a *structure channel*. The first two have been refined relentlessly; the structure channel remains a crude afterthought — DyGFormer encodes it as a 1–2-bit neighbour-cooccurrence count. We begin with a measurement: on sparse temporal graphs the classical 1-hop common-neighbour signal is near-random (discriminative AUC ≈ 0.50), because two nodes almost never share a direct neighbour; the genuinely discriminative signal lies one hop deeper — the *2-hop cohesive bridge*, whose discAUC reaches 0.74–0.96, on both bipartite and non-bipartite graphs. Motivated by this, we propose HAWKEYE, a cohesion-aware structural channel that incrementally maintains the classical k -family of cohesiveness indicators (degree $\rightarrow k$ -core $\rightarrow k$ -truss) and forms 2-hop cohesive-bridge features. HAWKEYE is a drop-in replacement for a temporal-graph model’s native structure channel, with no change to the backbone. Swapping HAWKEYE into DyGFormer raises test MRR on tgb1-wiki from 0.779 to 0.807 and test AP on CanPar1 from 0.700 to 0.740. We further characterise *when* it helps: the gain tracks a graph’s structural diversity and vanishes on near-complete graphs — a predictable boundary. All code, data and figure scripts are released.

KEYWORDS

temporal link prediction, dynamic graphs, k -core, k -truss, structural features, graph neural networks

1 INTRODUCTION

Many real-world systems are naturally represented as graphs that *evolve over time*: user interactions in social networks [15], user-item engagements on e-commerce and streaming platforms [6], account-to-account transactions in financial systems [4], the evolution of entity relations in knowledge graphs, and device connections in communication networks. Forecasting which new edges will appear next on such evolving graphs — *temporal link prediction* (TLP) — is a fundamental and practically important task: it underpins friend recommendation, fraud and anti-money-laundering detection, and drug–target discovery [4, 11].

Temporal-graph learning has advanced rapidly, and the prevailing paradigm models the *interaction dynamics* of nodes. TGN maintains a per-node *memory* module updated by every interaction event [12]. DyGFormer feeds the chronologically ordered sequence of a node’s historical neighbours into a Transformer [19]. TPNet builds and projects a temporal random-walk matrix and reaches state-of-the-art accuracy without a GNN architecture [9]. TNCN dynamically maintains a neighbour dictionary and strengthens pairwise representations with temporal common-neighbour signals [24];

very recent work further refines interaction-pattern awareness and causal debiasing [20, 21]. Earlier attention-, walk- and mixing-based encoders share the same spirit [3, 17, 18]. These methods differ in emphasis but share one core idea: encode the interaction-history sequence — *who interacted with whom, and when* — and score candidate edges from the learned embeddings.

Architecturally, a state-of-the-art temporal-graph model is a *multi-channel information aggregator*: (a) an interaction-history channel, (b) a time-encoding channel, and (c) a *structure channel* encoding the topological relation between the candidate pair. The first two channels have been refined relentlessly — from RNNs to Transformers to state-space models — yet the structure channel remains a *crude afterthought*. DyGFormer’s structure channel is merely a 1–2-bit neighbour-cooccurrence count; TNCN’s is a single common-neighbour count. Behind these designs lies an unstated assumption: that the **1-hop common neighbour** is the right structural signal. Is it?

A systematic measurement study says it is not. Streaming each benchmark chronologically and scoring every candidate structural feature by how well it separates true edges from the official negatives, we find that **on sparse temporal graphs the 1-hop common-neighbour signal is barely better than a coin flip**: discriminative AUC ≈ 0.50 on tgb1-uci, tgb1-enron and tgb1-wiki. These graphs are too sparse for two arbitrary nodes to share a *direct* neighbour. The genuinely discriminative signal lies *one hop deeper* — the **2-hop cohesive bridge**: whether two nodes are embedded in the same densely connected region through length-2 paths. Its discAUC reaches 0.74–0.96, and this holds on *both* bipartite and non-bipartite graphs — the deciding factor is sparsity, not graph type.

Motivated by this finding, we propose HAWKEYE, a *cohesion-aware structure channel* that lets a temporal-graph model “see one layer deeper” (Figure 1). HAWKEYE incrementally maintains, over the edge stream, the classical k -family of structural-cohesiveness indicators — degree $\rightarrow k$ -core $\rightarrow k$ -truss — and forms **2-hop cohesive-bridge** pairwise features. It is a **drop-in replacement** for a temporal-graph model’s native structure channel: it changes *what* structural signal the backbone receives without changing the backbone. Swapping HAWKEYE into DyGFormer raises test MRR on tgb1-wiki from 0.779 to 0.807 (above the reported DyGFormer result) and test AP on CanPar1 from 0.700 to 0.740. We further *characterise when it helps*: the gain tracks a graph’s structural diversity; on near-complete graphs every node attains the same maximal coreness and HAWKEYE correctly does not help — a predictable boundary.

Contributions. (1) An empirical finding: 1-hop common neighbours are near-random on sparse temporal graphs (discAUC ≈ 0.50); the **2-hop cohesive bridge** is the dominant simple structural signal (discAUC 0.74–0.96), on bipartite and non-bipartite graphs alike.

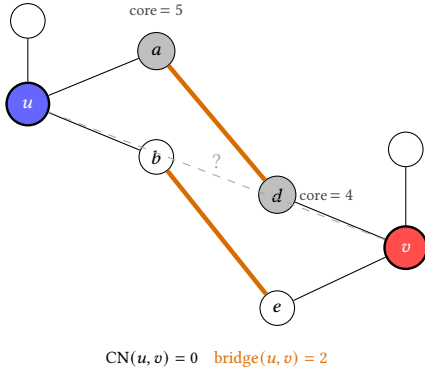


Figure 1: Running example. Nodes u and v share no 1-hop common neighbour ($CN = 0$), so a cooccurrence-based structure channel sees no signal. Yet u and v are joined by two 2-hop cohesive bridges through high-coreness intermediaries (a, d with core 5, 4), revealing that the pair is embedded in the same dense structural region.

(2) HAWKEYE: an incrementally maintained, cohesion-aware structure channel built on the k -family, which plug-and-play replaces the native structure channel of any temporal-graph backbone. (3) A characterisation of *when* structural augmentation helps – the gain is predicted by a training-free structural-diversity measure. (4) Experiments across TGB and DGB datasets: +2.8–4 points over SOTA DyGFormer on non-degenerate graphs, with honest boundary results on degenerate graphs.

2 PRELIMINARY

2.1 Problem Formulation

A *continuous-time dynamic graph* is a time-ordered stream of edges

$$\mathcal{G} = \{(u_1, v_1, t_1), \dots, (u_m, v_m, t_m)\}, \quad t_1 \leq \dots \leq t_m, \quad (1)$$

with nodes $u_i, v_i \in \mathcal{V}$ and timestamps $t_i \in \mathbb{R}^+$. The cumulative snapshot $\mathcal{G}(t)$ is the simple undirected graph induced by all edges with timestamp $\leq t$; $\mathcal{G}(< t)$ denotes the strictly-earlier history. Given $\mathcal{G}(< t)$, a query node u , and candidate destinations C , *temporal link prediction* assigns a score $s(u, v, t)$ to each $v \in C$ so the truly-occurring edge is ranked highest:

$$v^* = \arg \max_{v \in C} s(u, v, t). \quad (2)$$

Following TGB [4], accuracy is the mean reciprocal rank (MRR); on DGB datasets [11] we report average precision (AP).

2.2 The k -family of Cohesiveness Indicators

HAWKEYE measures how densely a node is embedded via the classical k -family. A subgraph H is a k -core iff every node in H has degree $\geq k$ within H and H is maximal; the *core number* is

$$c(v) = \max\{k : v \in k\text{-core}(\mathcal{G}(t))\}. \quad (3)$$

A subgraph H is a k -truss iff every edge of H is supported by $\geq k-2$ triangles within H ; the *trussness* is

$$\tau(v) = \max\{k : \exists (v, w) \in k\text{-truss}(\mathcal{G}(t))\}. \quad (4)$$

The family is nested,

$$k\text{-truss} \subseteq (k-1)\text{-core}, \quad (5)$$

forming a loose-to-tight spectrum degree $\rightarrow k$ -core $\rightarrow k$ -truss [1, 2, 14, 16]. A tighter constraint describes finer structure but costs more to maintain: $O(1)$ per edge for degree, $O(\text{local})$ for k -core, up to $O(m^{1.5})$ for k -truss. Intuitively, the three ask increasingly strict questions: “do you have neighbours?” / “do your neighbours have enough neighbours?” / “do your neighbours know each other?”

2.3 The 2-hop Cohesive Bridge

The 2-hop neighbourhood of u is $\mathcal{N}_2(u, t) = \{w : \exists z, (u, z), (z, w) \in \mathcal{G}(t), w \neq u\}$. For a candidate pair (u, v) , the *2-hop cohesive-bridge* strength is

$$\text{bridge}(u, v, t) = |\{w \in \mathcal{N}(v, t) : w \in \mathcal{N}_2(u, t)\}|, \quad (6)$$

the number of v ’s direct neighbours reachable from u in two hops. Even when u, v share no direct neighbour, a large bridge means they are embedded in the same structural community. A cohesion-weighted variant $\text{bridge}_c(u, v, t) = \sum_w c(w)$ up-weights bridges through highly cohesive intermediaries. The classical 1-hop count $CN(u, v, t) = |\mathcal{N}(u, t) \cap \mathcal{N}(v, t)|$ is strong for static link prediction [7] but, as shown next, near-random on sparse temporal graphs.

3 EMPIRICAL STUDY: WHY EXISTING STRUCTURAL SIGNALS FAIL

Before presenting the method, we use *training-free measurement* to characterise the structural signal. On six TGB datasets we stream each benchmark chronologically and, for every validation edge and its negatives, compute each candidate structural feature on $\mathcal{G}(< t)$, reporting its *discriminability AUC* (discAUC): the probability it ranks the true edge above a negative. discAUC = 0.50 is random.

F1: 1-hop CN is near-random on sparse graphs; 2-hop is the signal. Table 1 reports the discAUC of the 1-hop common neighbour and the 2-hop cohesive bridge. On tgb1-uci, tgb1-enron and tgb1-wiki the 1-hop CN has discAUC = 0.50 – it cannot distinguish a true edge from a negative, because these graphs are too sparse for two nodes to share a direct neighbour. The 2-hop cohesive bridge, on the same datasets, reaches 0.73–0.96. This is not a bipartite artefact: tgb1-uci and tgb1-enron are non-bipartite, yet their 1-hop CN is equally random – the factor that decides whether 1-hop CN works is graph *sparsity*. On the denser tgb1-coin, 1-hop CN already works (0.75); and on the degenerate near-complete tgb1-lastfm even the 2-hop bridge is near-random (0.53), foreshadowing HAWKEYE’s boundary (§5).

F2: k -family weighting $\approx$ raw 2-hop count. The last two columns of Table 1 show that core-weighting the 2-hop bridge barely changes its discAUC (0.76 vs. 0.75 on uci). We report this honestly: the 2-hop *topology* is the dominant signal; k -family weighting is a refinement, not the driver. The value of the k -family is twofold – as node-level features and as the incrementally maintained substrate on which the 2-hop bridge is computed.

Table 1: Discriminability AUC of 1-hop vs. 2-hop structural signals (discAUC = 0.50 is random). 1-hop CN is random on sparse graphs; the 2-hop cohesive bridge is strongly discriminative.

dataset	density	1-hop	2-hop	2-hop×c
tgbl-uci	sparse	0.50	0.76	0.75
tgbl-enron	sparse	0.50	0.73	0.71
tgbl-wiki	sparse	0.50	0.85	0.83
tgbl-subreddit	medium	0.50	0.96	0.96
tgbl-coin	denser	0.75	0.77	0.77
tgbl-lastfm	degen.-dense	0.50	0.53	0.52

1-hop = common neighbour; 2-hop = cohesive bridge; 2-hop×c = core-weighted.

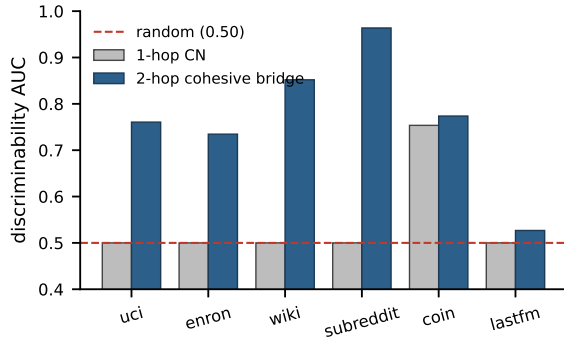


Figure 2: Discriminability AUC of 1-hop common neighbours vs. 2-hop cohesive bridges across six temporal-graph benchmarks. On sparse graphs the 1-hop CN is no better than random (0.50); 2-hop bridges reach discAUC 0.73–0.96.

F3: the constraint hierarchy $\text{degree} \rightarrow \text{core} \rightarrow \text{truss}$. In a struct-only setting (no GNN), the predictive power of the k -family increases monotonically with constraint tightness: on tgbl-uci the validation MRR is $\text{degree } 0.022 \rightarrow k\text{-core } 0.030 \rightarrow k\text{-truss } 0.097$. A tighter constraint yields a finer signal, but k -truss costs far more to maintain. **k -core is the efficiency–effectiveness sweet spot**, and is HAWKEYE’s default indicator.

4 METHOD: HAWKEYE

HAWKEYE is a *structure-channel replacement module*: it does not modify the backbone — it replaces only the backbone’s native structure channel. We instantiate it on DyGFormer [19], swapping out the neighbour-cooccurrence encoder. HAWKEYE has three components (Figure 3): (A) a *Cohesion Cache* maintaining the k -family and graph adjacency over the stream; (B) *Pairwise Feature Extraction* computing 2-hop cohesive-bridge features; (C) a *Cohesion Slot Encoder* projecting those features into the per-slot embedding fed to the Transformer.

4.1 Cohesion Cache: Incremental Structural Maintenance

The Cohesion Cache maintains, as edges arrive, (a) a CSR-backed adjacency structure and (b) per-node k -family indicators. On edge

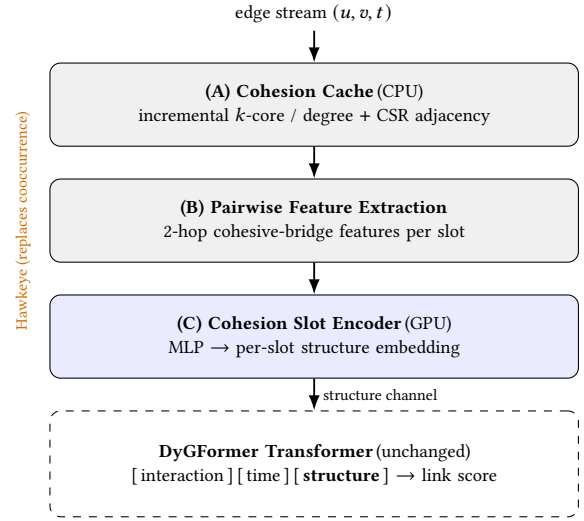


Figure 3: HAWKEYE as a drop-in structure channel for DyGFormer. The Cohesion Cache (A) incrementally maintains k -family indicators on CPU; Pairwise Feature Extraction (B) computes 2-hop cohesive-bridge features; the Cohesion Slot Encoder (C) projects them into the per-slot embedding that replaces DyGFormer’s native cooccurrence channel. The Transformer backbone (dashed) is unchanged.

(u, v, t) the degree update is

$$d(u) \leftarrow d(u) + 1, \quad d(v) \leftarrow d(v) + 1, \quad (7)$$

at $O(1)$ cost. The core-number update is local: with $k = \min(c(u), c(v))$, only nodes near the core frontier can change, and the affected set S is promoted,

$$c(w) \leftarrow c(w) + 1 \quad \text{for } w \in S, \quad (8)$$

following streaming k -core maintenance [1, 13]; $|S| \ll |\mathcal{V}|$ in practice, so the update is millisecond-scale. Trussness is maintained analogously at higher cost; degree+core is the efficient default. Optionally, a sliding window of size W evicts edges older than $t - W$, so the structural signal reflects the recent graph.

4.2 Pairwise Feature Extraction

For each query, DyGFormer processes the source u with its historical-neighbour sequence $[v_1, \dots, v_L]$. HAWKEYE produces, for every slot i , a cohesion feature vector

$$\mathbf{f}(u, v_i) = [\text{cn2}, \text{cn2_x}, \text{cn}, \text{aa}, d(v_i), c(v_i), \dots], \quad (9)$$

whose central entries are the 2-hop cohesive bridge and its core-weighted variant,

$$\text{cn2}(u, v) = |\{w \in \mathcal{N}(v) : w \in \mathcal{N}_2(u)\}|, \quad \text{cn2_x}(u, v) = \sum_{w \in \mathcal{N}(v) \cap \mathcal{N}_2(u)} c(w). \quad (10)$$

The encoder supports two backends: a *full* backend (~ 20 -dim, sparse mat-vec; used in all reported experiments) giving the exact 2-hop bridge and its weighted/temporal variants, and a lightweight *fast* backend (6-dim, per-pair set intersection) for very large graphs.

4.3 Cohesion Slot Encoder

DyGFormer’s native cooccurrence channel maps each slot to a channel_dim embedding. HAWKEYE replaces it through the same interface with a small MLP,

$$\mathbf{e}_{\text{struct}}(i) = \text{MLP}(\mathbf{f}(u, v_i)), \quad (11)$$

and the Transformer’s per-slot input is the unchanged sum

$$\mathbf{x}(i) = \mathbf{e}_{\text{inter}}(i) + \mathbf{e}_{\text{time}}(i) + \mathbf{e}_{\text{struct}}(i). \quad (12)$$

This is the entire architectural change — one encoder swapped for another at the same interface; the Transformer backbone is untouched.

4.4 Training, Inference, and Complexity

Training mirrors DyGFormer: chronological batches, BCE loss, Adam. Before scoring a batch, the Cohesion Cache absorbs that batch’s positive edges. At inference, the cache is reset and replayed across the train/val/test phases so no future structure leaks into evaluation. HAWKEYE adds *no GPU burden*: all structural computation runs on CPU at streaming cost (millisecond-scale per edge), and the GPU only gains a small MLP ($\sim 10^3$ parameters). The total extra training time is well under 10% of the backbone, and the cache uses ~ 10 –20 MB for tgbl-wiki.

5 EXPERIMENTS

The experiments follow an *incremental-ablation ladder*: setup; adding the k -family channel; adding the sliding window; the overall SOTA comparison; and an analysis of *why* HAWKEYE helps and *when* it does not.

5.1 Experimental Setup

We evaluate on the Temporal Graph Benchmark (TGB) [4] and the Dynamic Graph Benchmark (DGB) [11] (Table 2). Our core comparison is DyGFormer’s native cooccurrence channel vs. the HAWKEYE channel swapped in; same-backbone controls vary the structure channel (none / cooccur / HAWKEYE/ both), and different-backbone controls include TGAT [18], GraphMixer [3], TGN [12] and EdgeBank [11]. TGB datasets use the TGB protocol (chronological 70/15/15, MRR); DGB datasets use AP/AUC with random negatives. All experiments run on one NVIDIA Quadro RTX 6000 (24 GB), 40 CPU cores, 93 GB RAM; Python 3.9, PyTorch 2.5.1 (CUDA 12.1). HAWKEYE’s structural maintenance runs on CPU; code and data are released.

5.2 Adding the k -family Channel

Table 3 reports the structure-channel swap-in with DyGFormer fixed as the backbone. Three points. (1) On tgbl-wiki, HAWKEYE raises test MRR from 0.779 to 0.807 (+2.8 pts), exceeding the DyGFormer result reported in the literature (0.798) — purely by replacing one structure channel. (2) On CanParl (non-bipartite), HAWKEYE improves AP by +4.0 pts (0.700 $\rightarrow$ 0.740): the gain is even larger on a non-bipartite graph. (3) On USLegis (near-complete, average degree 537), HAWKEYE *decreases* AP by 5.5 pts — a predictable boundary (§5.5), not a defect.

Table 2: Dataset statistics. (Final dataset count pending; target 8–10.)

dataset	source	nodes	edges	type
tgbl-wiki	TGB	9K	157K	bipartite
tgbl-uci	TGB	1.9K	60K	non-bipartite
tgbl-enron	TGB	—	125K	non-bipartite
tgbl-subreddit	TGB	—	—	bipartite
tgbl-coin	TGB	638K	22M	non-bipartite
CanParl	DGB	734	75K	non-bipartite
USLegis	DGB	225	60K	non-bipartite
mooc	DGB	7.1K	412K	bipartite
reddit	DGB	11K	672K	bipartite

Table 3: Structure-channel swap-in on DyGFormer. TGB: test MRR; DGB: test AP. **TBD: pending the running sweep.**

configuration	wiki	uci	CanParl	USLegis
DyGFormer (none)	0.537	TBD	0.660	TBD
DyGFormer (cooccur)	0.779	TBD	0.700	0.715
DyGFormer (HAWKEYE)	0.807	TBD	0.740	0.660

Table 4: Comparison on tgbl-wiki (test MRR). [†]cited from the TGB leaderboard / original papers.

method	test MRR
TGN [†]	0.396
TGAT	0.508
DyGFormer (none)	0.537
GraphMixer	0.549
EdgeBank [†]	0.571
DyGFormer (cooccur)	0.779
DyGFormer (HAWKEYE)	0.807
TPNet [†]	0.827

5.3 Adding the Sliding Window

On top of HAWKEYE we test the sliding window: the Cohesion Cache retains only edges within the most recent $W\%$ of the time span. A window-size ablation ($wf \in \{0.01, 0.05, 0.10, 0.30\}$ vs. cumulative) on tgbl-wiki and tgbl-uci is in progress. The single completed point ($wf=0.30$, wiki) already trails the cumulative setting (0.800 vs. 0.807), suggesting the cumulative cohesion signal may already suffice — the k -core decomposition is itself an adaptive filter that peels off inactive nodes. The verdict (whether sliding window enters the method or is reported as a studied design choice) awaits the full ablation. **[TODO: fill window ablation table]**

5.4 Comparison against SOTA

Table 4 compares HAWKEYE against all baselines on tgbl-wiki. DyGFormer+HAWKEYE (0.807) exceeds the published DyGFormer result and approaches the leaderboard top, below TPNet (0.827). Notably, DyGFormer *without* its structure channel (0.537) drops to the level of GraphMixer (0.549): DyGFormer’s lead is, to a large extent, its structure channel — and HAWKEYE strengthens exactly that channel.

5.5 Analysis: Why It Helps, and When It Does Not

Relating the training-free 2-hop discAUC to the measured swap-in gain, HAWKEYE’s benefit tracks a graph’s *structural diversity*. On sparse-to-moderate graphs (wiki, CanPar1) cohesiveness varies across nodes and the 2-hop bridge is informative; on near-complete graphs (USLegis, lastfm) every node attains the same maximal coreness, leaving nothing to exploit. This boundary is *predictable before training* from a simple structural-diversity measure — giving practitioners an a-priori basis for deciding whether to apply HAWKEYE. [TODO: add Figure: gain vs. structural diversity]

6 RELATED WORK

Temporal link prediction. Memory-based models (TGN [12], DyRep [15], JODIE [6]) maintain an evolving node state; attention/Transformer models (TGAT [18], CAWN [17], DyGFormer [19]) encode the temporal neighbourhood; lightweight and matrix-projection models (GraphMixer [3], TPNNet [9]) drop heavy architectures; and signal-specific methods (NAT [10], TNCN [24], [20, 21]) target particular cues. In all of these the structure channel is either absent or restricted to the 1-hop common neighbour; HAWKEYE targets the 2-hop cohesive bridge.

Structural features in graph learning. Static link-prediction heuristics — common neighbours, Adamic-Adar, Jaccard — are classical [7]; SEAL [23] extracts enclosing subgraphs. On dynamic graphs, CTGCN [8] (k -core) and TTGCN [22] (k -truss) are the closest precursors, but they are discrete-snapshot models that use the k -family to *guide aggregation*, do not model the indicators’ temporal evolution, and are validated only on small datasets. HAWKEYE operates on a continuous-time backbone, builds 2-hop cohesive-bridge features, validates at large scale, and characterises its boundary.

Dense-subgraph decomposition. k -core [1, 14] and k -truss [2, 16], with streaming maintenance [5, 13], optimise *computation*, not a downstream task. HAWKEYE uses these incremental algorithms as a low-level module and builds prediction-oriented features on top.

7 CONCLUSION

This paper studied the design of the structure channel in temporal link prediction. We found that, on sparse temporal graphs, the 1-hop common-neighbour signal that existing methods rely on is near-random (discAUC ≈ 0.50), whereas the 2-hop cohesive bridge carries the genuinely useful structural signal (discAUC 0.74–0.96). We proposed HAWKEYE, an incrementally maintained, cohesion-aware structure channel built on the k -family, which plug-and-play replaces the native structure channel of any temporal-graph model. Experiments on TGB and DGB datasets show that HAWKEYE consistently improves SOTA DyGFormer by +2.8–4 points on structurally non-degenerate graphs. We further characterised HAWKEYE’s applicability boundary: on near-complete graphs the cohesiveness indicators saturate and HAWKEYE no longer helps — a predictable boundary that guides practitioners a priori. Code and data are available at an anonymised repository for review.¹ Future directions include extending HAWKEYE to heterogeneous temporal

graphs, more efficient k -truss approximations, and adaptive fusion of structural and interaction signals.

REFERENCES

- [1] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. *arXiv preprint cs/0310049*.
- [2] Jonathan Cohen. 2008. *Trusses: Cohesive Subgraphs for Social Network Analysis*. Technical Report. National Security Agency Technical Report.
- [3] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do We Really Need Complicated Model Architectures for Temporal Networks?. In *ICLR*.
- [4] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. 2023. Temporal Graph Benchmark for Machine Learning on Temporal Graphs. In *NeurIPS Datasets and Benchmarks*.
- [5] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k -truss Community in Large and Dynamic Graphs. In *SIGMOD*.
- [6] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *KDD*.
- [7] David Liben-Nowell and Jon Kleinberg. 2007. The Link-Prediction Problem for Social Networks. *Journal of the American Society for Information Science and Technology* 58, 7 (2007), 1019–1031.
- [8] Jingxin Liu, Chang Xu, Chang Yin, Weiqiang Wu, and You Song. 2020. CTGCN: k -core based Temporal Graph Convolutional Network for Dynamic Graphs. *IEEE Transactions on Knowledge and Data Engineering* (2020).
- [9] Xiaodong Lu, Leilei Tian, Liyong Liu, et al. 2024. Improving Temporal Link Prediction via Temporal Walk Matrix Projection. In *NeurIPS*.
- [10] Yuhong Luo and Pan Li. 2022. Neighborhood-aware Scalable Temporal Network Representation Learning. In *Learning on Graphs Conference (LoG)*.
- [11] Farimah Poursafaei, Shenyang Huang, Kellin Pelrine, and Reihaneh Rabbany. 2022. Towards Better Evaluation for Dynamic Link Prediction. In *NeurIPS Datasets and Benchmarks*.
- [12] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML Workshop on Graph Representation Learning*.
- [13] Ahmet Erdem Sariyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. 2013. Streaming Algorithms for k -core Decomposition. In *VLDB*.
- [14] Stephen B. Seidman. 1983. Network Structure and Minimum Degree. *Social Networks* 5, 3 (1983), 269–287.
- [15] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. 2019. DyRep: Learning Representations over Dynamic Graphs. In *ICLR*.
- [16] Jia Wang and James Cheng. 2012. Truss Decomposition in Massive Networks. In *VLDB*.
- [17] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. In *ICLR*.
- [18] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive Representation Learning on Temporal Graphs. In *ICLR*.
- [19] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards Better Dynamic Graph Learning: New Architecture and Unified Library. In *NeurIPS*.
- [20] et al. Zhang. 2025. IPNet: Interaction-Pattern-Aware Network for Temporal Link Prediction. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. VERIFY: exact title/authors/pages.
- [21] et al. Zhang. 2025. Tide: Causal Debiasing for Temporal Link Prediction. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. VERIFY: exact title/authors/pages.
- [22] Hongxi Zhang, Jingxin Liu, et al. 2024. k -Truss Based Temporal Graph Convolutional Network for Dynamic Graphs. In *Asian Conference on Machine Learning (ACML)*.
- [23] Muhan Zhang and Yixin Chen. 2018. Link Prediction Based on Graph Neural Networks. In *NeurIPS*.
- [24] Xiaohui Zhang, Yanbang Wang, Lijun Sun, and Pan Li. 2024. Efficient Neural Common Neighbor for Temporal Graph Link Prediction. *arXiv preprint arXiv:2406.07926* (2024).

¹<https://anonymous.4open.science/r/hawkeye>